

09 장 학습 과제

과제 1. 책임이 집중된 클래스 분석

입력 처리, 이동, 공격, 체력, 저장, HUD 출력, 사운드 재생을 모두 담당하는 **Player** 클래스를 작성한다. 클래스의 데이터 멤버와 멤버 함수를 책임별로 분류하고 서로 다른 변경 이유를 최소 네 가지 이상 찾는다.

분석 결과를 바탕으로 클래스를 분리하고 **Mermaid** 클래스 다이어그램으로 관계를 표현한다. 각 클래스의 책임을 한 문장으로 설명할 수 있어야 한다.

과제 2. 데이터와 행위의 배치

Character가 **GetHp()**와 **SetHp()**만 제공하고 외부 함수가 피해와 회복을 계산하는 코드를 작성한다. 여러 외부 함수에서 체력 범위 보정이 반복되는 문제를 확인한다.

체력 상태와 규칙을 **HealthComponent**로 이동하여 구조를 개선한다. 아래 불변식을 모든 공개 함수가 유지해야 한다.

```
0 <= hp <= maxHp
maxHp > 0
```

피해량과 회복량이 0 이하일 때의 처리 규칙도 명확히 정한다.

과제 3. 응집도와 결합도 개선

CombatService가 대상의 체력을 직접 읽고 계산하며 콘솔에 결과를 출력하는 코드를 작성한다. 체력 처리와 기록 출력을 각각 **IDamageable**, **ICombatLog** 역할로 분리한다.

개선 전후 코드에서 전투 공식, 체력 저장 방식, 출력 대상이 바뀔 때 수정해야 하는 클래스를 비교한다. 변경 영향 범위를 표로 정리한다.

과제 4. 최소 인터페이스와 불변식

인벤토리 클래스를 설계한다.

필수 기능:

- 아이템 추가
- 아이템 제거
- 식별자로 검색
- 보유 개수 확인
- 최대 슬롯 수 유지
- 중복 허용 여부 선택

내부 **std::vector**를 직접 반환하거나 수정 가능한 참조를 공개하지 않는다. 추가와 제거 실패는 열거형 또는 결과 객체로 표현한다. 클래스가 유지해야 할 불변식을 문서로 작성한다.

과제 5. 개방-폐쇄 원칙과 공격 행동

근접, 원거리, 마법 공격을 열거형과 **switch**문으로 구현한다. 독 공격을 추가하면서 수정한 위치를 기록한다.

공격 계산을 **IAttackBehavior**로 분리하고 각 공격을 구체 행동 객체로 구현한다. **Player**가 **std::unique_ptr<IAttackBehavior>**를 소유하고 실행 중 공격 방식을 교체할 수 있도록 만든다.

상속으로 **MeleePlayer**, **RangedPlayer**, **MagicPlayer**를 만드는 방식과 행동 객체를 구성하는 방식을 클래스 수, 실행 중 교체, 재사용 관점에서 비교한다.

과제 6. 리스코프 치환 원칙 검사

IMovable 인터페이스와 **Player**, **Monster**, **StaticDecoration** 클래스를 구성한다. 처음에는 세 클래스가 모두 **IMovable**을 구현하도록 작성하고 **StaticDecoration::MoveTo()**가 아무 작업도 하지 않도록 만든다.

기반 타입을 사용하는 시험 함수를 작성하여 치환 문제가 드러나게 한다. 위치 상태를 가진 **GameObject**와 이동 역할인 **IMovable**을 분리하여 계층을 개선한다. 사전 조건, 사후 조건, 불변식 관점에서 수정 이유를 설명한다.

과제 7. 인터페이스 분리

Update(), **Render()**, **ProcessInput()**, **TakeDamage()**, **Save()**를 포함하는 큰 **IGameEntity**를 작성한다. **Player**, **Wall**, **ParticleEffect**에서 구현하면서 사용하지 않는 함수가 생기는지 확인한다.

인터페이스를 역할별로 분리한다.

- **IUpdatable**
- **IRenderable**
- **IInputReceiver**
- **IDamageable**
- **ISavable**

각 구체 클래스가 필요한 역할만 구현하도록 수정하고 **UML** 클래스 다이어그램으로 표현한다.

과제 8. 의존성 주입과 시험

치명타 확률이 있는 전투 서비스를 작성한다. 처음에는 **std::mt19937**과 **std::cout**을 서비스 내부에서 직접 사용한다.

난수와 기록 기능을 **IRandomSource**, **ICombatLog**로 분리하고 생성자 주입을 적용한다. 시험용 **FixedRandomSource**와 **MemoryCombatLog**를 만들어 이런 항목을 검증한다.

- 치명타 발생 시 피해량
- 일반 공격 피해량
- 이미 사망한 대상에 대한 처리
- 기록 메시지 개수
- 실제 체력보다 큰 피해를 가했을 때의 실제 감소량

과제 9. 전역 상태 제거

전역 객체 또는 싱글턴으로 난수, 사운드, 기록 기능을 제공하는 작은 프로그램을 작성한다. 각 함수가 실제로 어떤 전역 기능에 의존하는지 목록으로 정리한다.

전역 접근을 제거하고 필요한 객체에 생성자 또는 함수 매개변수로 의존성을 전달한다. 객체 생성과 연결을 `main()`에 모으고 초기화 순서와 수명 관계가 코드에 어떻게 드러나는지 설명한다.

과제 10. 객체지향 설계 종합

플레이어와 몬스터가 서로 공격하는 전투 프로그램을 구성한다.

필수 요구 사항:

- 체력은 `HealthComponent`가 관리
- 공격 계산은 교체 가능한 행동 객체로 구성
- 근접 공격, 원거리 공격, 마법 공격 구현
- 난수 공급과 전투 기록은 인터페이스로 분리
- 전역 상태 사용 금지
- 소유권은 값, 참조, `unique_ptr` 중 의도에 맞는 타입으로 표현
- 시험용 난수와 기록 객체 제공
- 구체 타입을 검사하는 `dynamic_cast` 사용 금지
- Mermaid 클래스 다이어그램과 공격 시퀀스 다이어그램 작성

구현이 끝나면 각 클래스의 책임, 변경 이유, 의존 객체, 소유권을 표로 정리한다.

인터페이스를 추가하지 않아도 되는 구체 클래스가 무엇인지 설명하고, 과도한 추상화를 피하기 위해 선택한 부분도 함께 기술한다.